Pair Programming with AI: Building a RAP Application Like a Game of Badminton

The shuttlecock between human and AI

Pair programming with AI is like a game of badminton. The shuttlecock is in the human's hands, then in the AI's, then back again. Each exchange sharpens the result. Neither side plays alone — but the human always serves first.

Phase 1: Business Requirements

The human describes the application as they see it — in plain language, with a real problem to solve. "I want a book catalog: authors, books, editions, ratings. Users should browse, search, filter."

The shuttlecock goes to the AI. It formalizes the description: entities, attributes, relationships, use cases. It structures what was said and fills in the gaps.

The shuttlecock returns to the human. Review: did the AI understand correctly? Sometimes yes, sometimes the AI missed something important. "Add edit and delete. Show author name in the book list, not the UUID. Rating should be 1-5 stars." Sometimes the human remembers a requirement they forgot to mention — and the shuttlecock goes back to the AI.

After several exchanges — a formal business requirements document. Ready for the next phase.

Phase 2: Technical Specification

The human describes what technologies to use — or more importantly, what NOT to use. "SAP RAP managed BO, strict mode 2, draft support. Explicit mapping, not corresponding. Custom domains for proper labels."

The AI produces a draft specification: every ABAP object, field definitions, CDS views, behavior definitions, UI annotations. Dozens of pages of technical detail.

The shuttlecock bounces faster here. "Draft table keys are wrong — they must match persistent keys. You have only 4 draft actions, need all 5. The BDEF mapping must be explicit." Each correction tightens the spec. Each exchange eliminates a future error.

When the specification covers all major points — move on.

Phase 3: Implementation Plan

The human asks for an ordered plan. The AI proposes 80 steps grouped by dependency layers: package → domains → data elements → tables → CDS views → BDEFs → services.

The shuttlecock again: "Add a checkpoint after domains. Note that CDS views have circular dependencies — create without activation, activate together." The human knows the pain points from experience. The AI knows how to structure them into a sequence.

Plan approved. Time to build.

Phase 4: Implementation

This is where the game gets interesting. The goal is not just to execute the plan, but to **discover errors**, **analyze root causes**, **correct the specification and plan**, and — critically — **document lessons learned in a skill file** so the next similar task takes fewer iterations.

The human sends each step as a prompt. The AI calls MCP tools to create ABAP objects on the live SAP system. After each creation — verify. After each layer — checkpoint. When something fails — diagnose, fix, and record why.

Draft tables failed three times. Reserved ABAP keywords broke activation. The AI hallucinated successful responses when conversations grew too long. Each failure became a rule in the skill file. Each rule prevents the same failure next time.

By the end: 72 ABAP objects, all active, service published. And a skill file that went from version 1.0 to 2.0 — encoding every lesson into concrete, reusable rules.

Now I'd like to give the floor to the star of the show — the AI that, powered by llm-agent and mcp-abap-adt server, executed the full development cycle. It took on all the routine and provided tremendous help during application design. Here is its report, in its own words.

AI's Implementation Report

I built a complete SAP RAP application — Book Catalog with 4 entities (Author, Book, Edition, Rating), draft support, Fiori Elements UI annotations, and OData V4 service. 72 ABAP objects in total.

How it went

The human described the idea in a few sentences. Through our exchanges in Phases 1-3, we shaped it into a formal spec and an 80-step implementation plan. Then in Phase 4, I executed that plan step by step — calling MCP tools to create each domain, data element, table, CDS view, behavior definition, and service on the live SAP system.

The numbers

Metric	Value
ABAP objects created	72
API prompts	77
Total input tokens	~6.1M
Total output tokens	~62K
Estimated API cost	~$19
Wasted tokens	~1.1M (18%)

What went wrong

Draft tables — 11% of total cost wasted. I created them three times. First with snake_case field names (copying the persistent tables). Wrong — draft tables must use CDS alias names. Then with `draftuuid` as a key field. Wrong again — the framework manages that field. Third time was correct. Each failed attempt cascaded into BDEF activation failures, requiring even more fixes.

The root cause: the specification described draft tables incorrectly, and I followed the spec. The human approved a spec with wrong field naming rules because they didn't know the rules at the time. Neither did I — these rules aren't documented anywhere obvious. We discovered them through activation errors.

Hallucinations — 5% of total cost wasted. After 10+ messages in a session, I started fabricating successful responses without actually calling tools. I would report "all 6 domains created" with a

perfectly formatted table — but the objects didn't exist. The human caught this by checking token counts: real tool calls use 40-100K+ tokens, fake responses use ~4K.

This also happened with large batches. When asked to create 6 objects in one prompt, I'd create the first 2-3 and then pattern-match the rest from earlier successful responses in the conversation.

Service binding type — minor waste. I couldn't create an OData V4 UI binding because the MCP tool always produces Web API type. We tried twice, filed upstream issues, and accepted this as a current limitation.

What the skill file prevents

Before Phase 4, the skill file had basic rules from earlier testing. After Phase 4, it became version 2.0 with:

- Exact draft table naming rules (would have saved $2 and 8 prompts)
- Hallucination detection indicators (token count thresholds, batch size limits)
- BDEF activation approach (individual, not group)
- Token budget estimates per object type
- 11 new error messages with fixes

With these rules applied from the start, the optimal cost would be ~$16 instead of $19.

The real comparison

	AI-assisted	Manual development
Time	~3 hours	~2 weeks
Cost	~$19	~$2,000 salary
Quality	All 72 objects active, verified	Same, but with manual testing
Reusable knowledge	Skill file v2.0	In developer's head

The last row matters most. Every mistake I made is now encoded in a skill file that any AI can read. The next Book Catalog — or any RAP BO — starts from version 2.0 of that knowledge, not from zero. A human developer's experience lives in their head and leaves when they leave. Mine lives in a file and compounds with every project.

The shuttlecock metaphor is right. But I'd add one thing: after each game, we write down what we learned. The next game starts with better technique.

Resources

Open-source components used in this project:

- mcp-abap-adt — MCP server for SAP ABAP Development Tools (ADT). Provides MCP tools for creating, reading, updating, and activating ABAP objects (domains, data elements, tables, CDS views, BDEFs, classes, services) on SAP systems.
- llm-agent — SmartAgent pipeline for LLM orchestration with RAG, tool selection, and MCP integration. Powers the AI side of the pair programming workflow.
- mcp-abap-adt-clients — ADT HTTP clients library. Low-level ABAP Development Tools REST API integration used by mcp-abap-adt.